Flock

The independent event creator

Group 11 · DH2643 · 2026-10-21

Alice Cohen, Elinor Selinder, Felix Larsson, Sandra Kåhre, Tilde Tärnvik



Agenda

1. Live demo
2. How it's built
3. Look inside code base
4. Technologies
5. Questions?

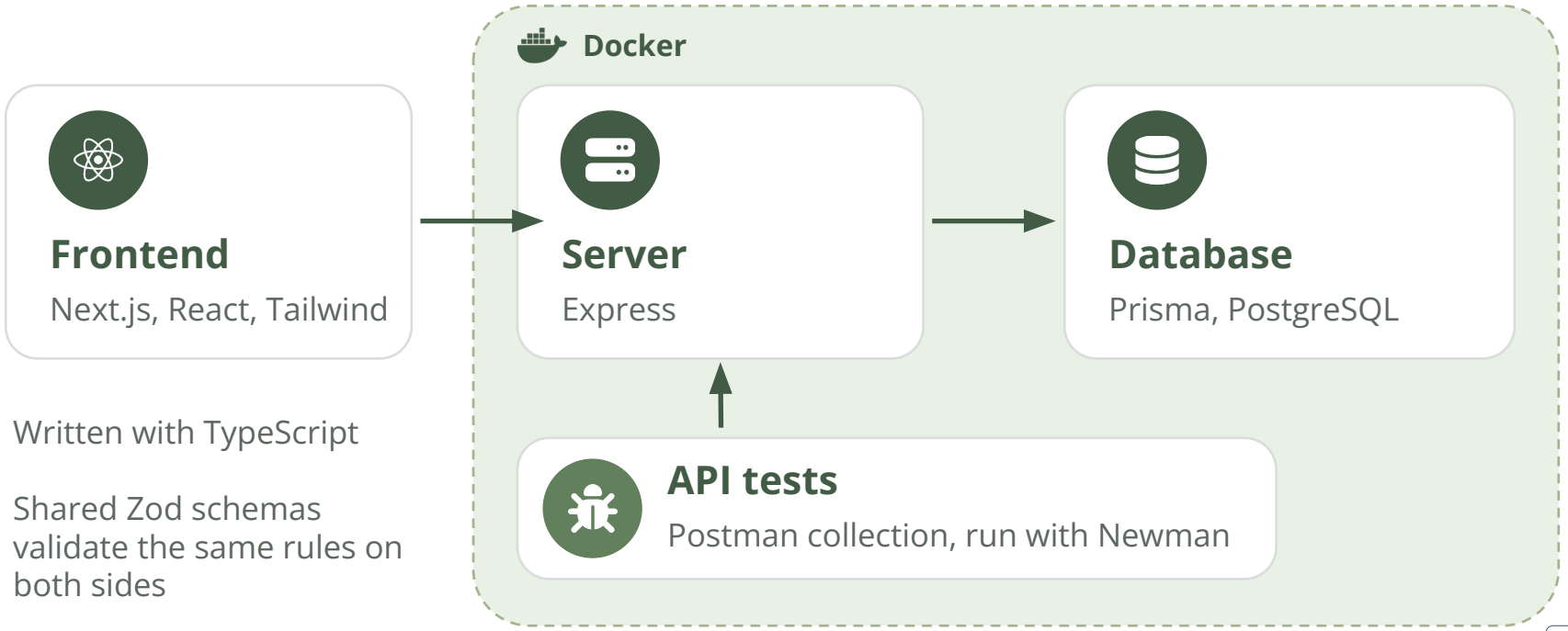
Live Demo

Let's look at our Project



HOW IT'S BUILT

Next.js frontend, Express API in Docker

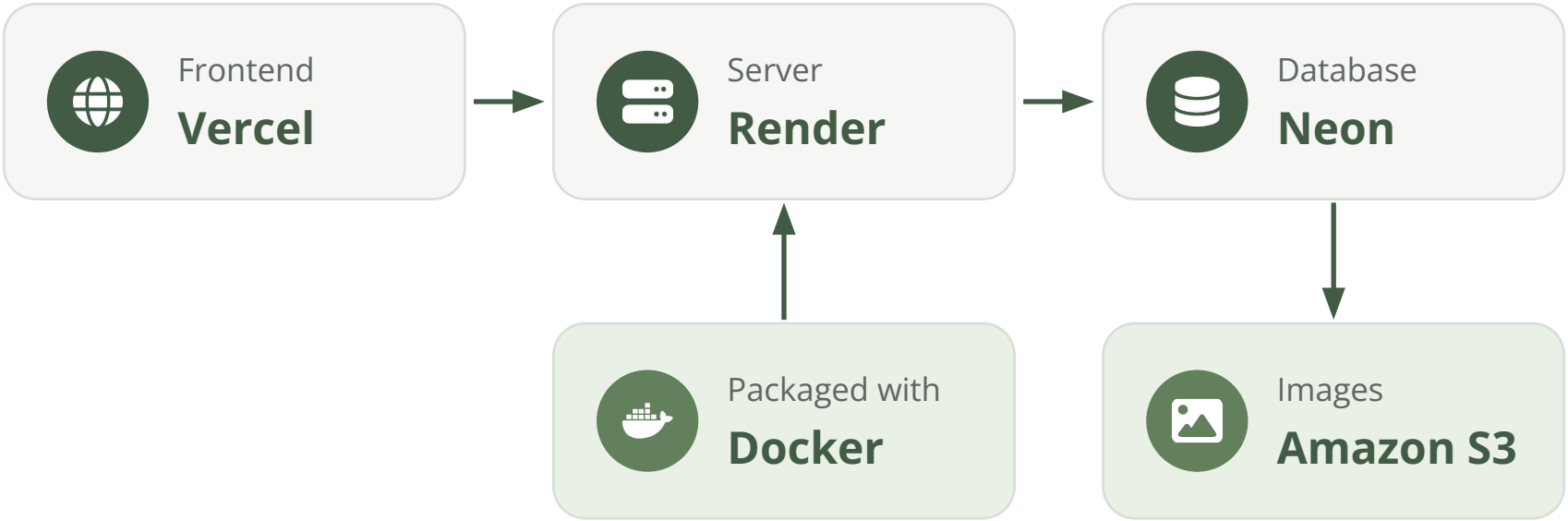


Written with TypeScript

Shared Zod schemas validate the same rules on both sides

HOSTING

Each part of Flock runs on its own host



Architecture

Let's take a quick look in the codebase





```
flock/
│
├─ shared config and root files
│
├─ frontend/                  # Frontend
│  ├─ public/                 # Static assets
│  │
│  ├─ src/
│  │  ├─ app/                 # Next.js App routing
│  │  │  ├── (protected)/     # Private pages protected by auth
│  │  │  └─ login/
│  │  │
│  │  ├─ features/            # Feature components, hooks, etc.
│  │  │  └─ event/            # etc.
│  │  │     ├── components/
│  │  │     ├── hooks/
│  │  │     └── utils/
│  │  │
│  │  └─ shared/              # Shared components, hooks, etc.
│  │     ├── components/
│  │     ├── hooks/
│  │     ├── types/
│  │     └── utils/
│  │
│  └─ config and root files   # node_modules, .next, etc.
│
```





```
│
├─ backend/                  # Backend
│   ├─ api-tests/            # Postman tests
│   │
│   ├─ prisma/
│   │  ├─ migrations/        # SQL migration history
│   │  └─ schema.prisma      # Data model + generator/datasource config
│   │
│   ├─ server/
│   │  ├─ controllers/       # Handles HTTP requests and validations
│   │  ├─ middleware/        # e.g. auth, error handling, image uploads
│   │  ├─ routes/            # Express route handlers
│   │  ├─ services/          # Database communication and business logic
│   │  ├─ types/
│   │  └─ utils/
│   │
│   └─ index.ts              # App entrypoint
│
└─ shared/
    └─ schemas/              # zod schemas and inferred types
```



Technologies

A deeper look at how flock is built

AUTHENTICATION

Logins use a JWT in an httpOnly cookie





On login

1. User logs in
2. User id goes in the JWT payload
3. JWT is signed with a secret
4. JWT is stored in an httpOnly cookie



Each request

1. Middleware reads the cookie
2. Verifies the signature and reads the user id
3. Looks up the user in the database
4. Adds the user id to the request



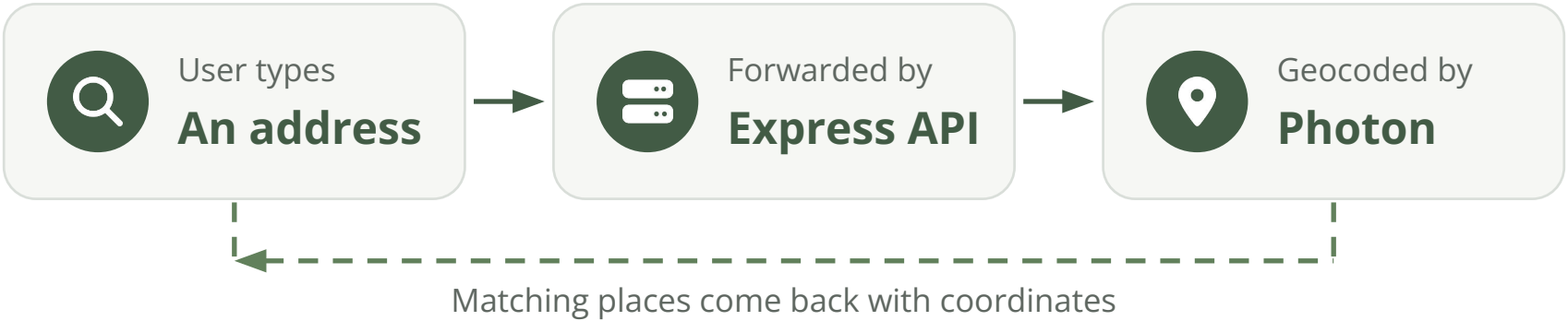
In the frontend

- AuthProvider wraps the whole app
- Calls /me to get the user
- Keeps the latest user state



EXTERNAL API

Address search uses Photon through our API



Open source, by Komoot

Built on OpenStreetMap

Free, no API key needed

IMAGE STORAGE

Images are stored in an AWS S3 bucket



Upload

Browser sends
The image

→

Received by
Express API

→

Stored in
S3 bucket

Why the upload goes through the API

AWS keys stay on the server
The browser never sees the bucket credentials.

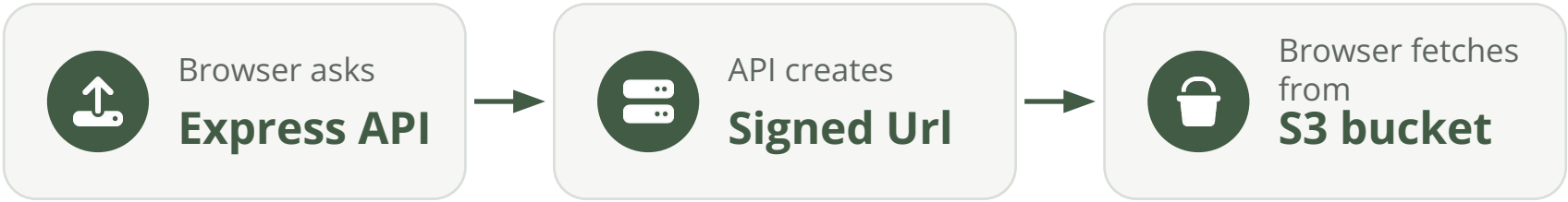
One place to check files
The API can validate an image before it is stored.



IMAGE STORAGE

Images are stored in an AWS S3 bucket

Displaying the image



How are the signed links more secure?

Links expire quickly

S3 rejects the URL after a set amount of time, or if anyone edits it.



Thank you

Questions?